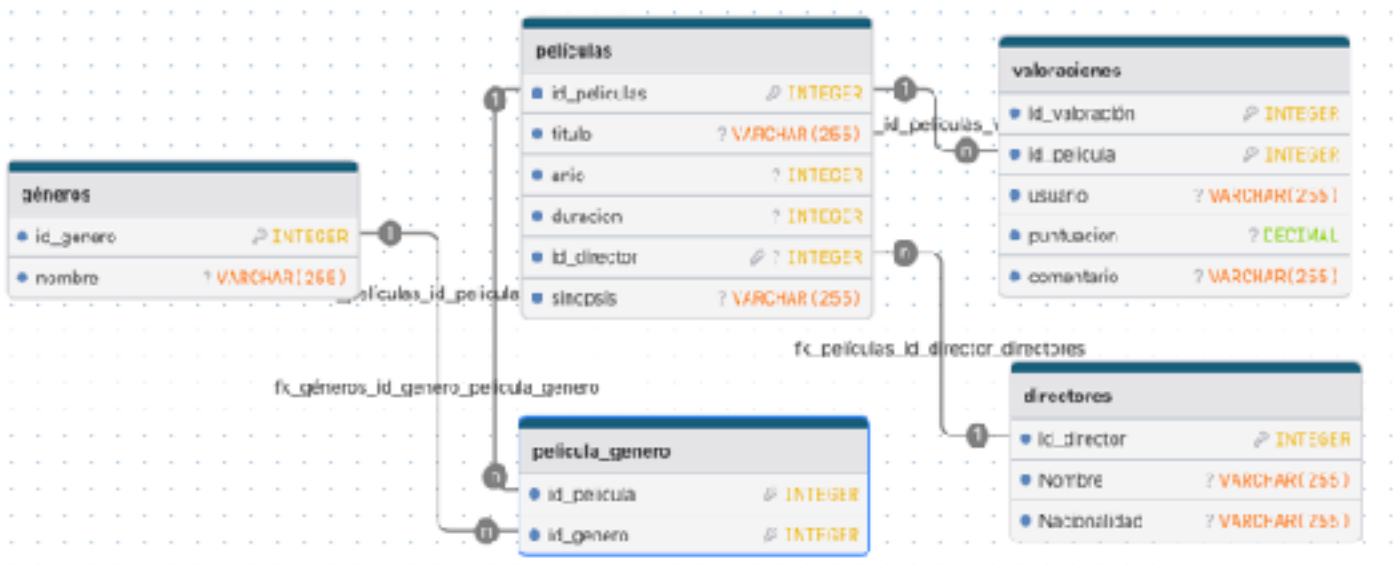


ACCESO A DATOS

El proyecto es una pequeña base de datos de películas de fantasía y ciencia ficción para visualizar en consola.

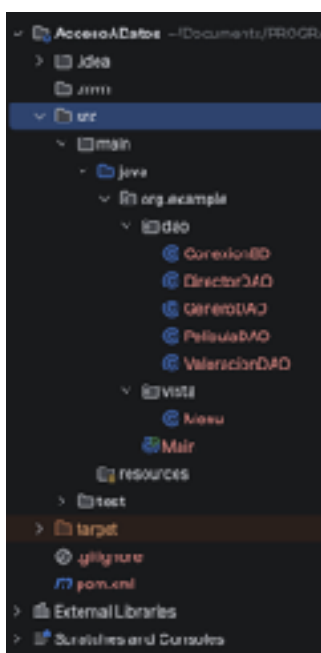
BASE DE DATOS

- La base de datos está ordenada en 5 tablas:



la tabla película_genero es una tabla intermedia con clave compuesta para solucionar la relación N:N entre película y género.

CODIGO APP



Respecto al código de la aplicación, está planteada con una separación entre:

- carpetas DAO (objetos de acceso a datos) que se encargan de la conexión a la base de datos y todas las operaciones CRUD que se realizan en ella.

- vista: Menu.java se encarga de la lógica general, de los métodos que gestionan el menú principal, piden datos y organizan la navegación.

Con esto ponemos dos capas separadas: la que ve el usuario y con la que interacciona, y la que gestiona los datos con la BD.

PRIMER PASO:

Conexión con la base de datos:

En el archivo ConexionDB.java se configura la llamada a la base de datos:

```
1 package org.example.dao;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 public class ConexionBD { 15 usages
8
9     private static final String URL = "jdbc:postgresql://localhost:5432/Cinefrikid8"; 1 usage
10    private static final String USUARIO = "iCarlos"; 1 usage
11    private static final String PASSWORD = ""; 1 usage
12
13    public static Connection obtenerConexion() throws SQLException { 14 usages
14        return DriverManager.getConnection(URL, USUARIO, PASSWORD);
15    }
16 }
```

Y en el Main se realiza la conexión y se lanza el menu:

```
1 package org.example;
2
3 import org.example.dao.ConexionBD;
4 import org.example.vista.Menu;
5
6 import java.sql.Connection;
7
8 public class Main {
9     public static void main(String[] args) {
10
11         try (Connection _ = ConexionBD.obtenerConexion()) {
12             System.out.println("Conexión correcta con PostgreSQL.");
13         } catch (Exception e) {
14             System.out.println("Error al conectar con la base de datos.");
15             System.out.println(e.getMessage());
16             return;
17         }
18
19         Menu menu = new Menu();
20         menu.mostrar();
21     }
22 }
```

Al ejecutar la aplicación, aparece el menú principal:

```
===== CINEPRIMI MANAGER =====
1. Listar películas
2. Añadir película
3. Eliminar película
4. Modificar película
5. Listar directores
6. Listar géneros
7. Listar valoraciones
8. Añadir valoración
9. Búsqueda avanzada
10. Top películas valoradas
0. Salir
Seleccione una opción:
```

Está configurado como un bucle do-while en el que se imprimen por consola las opciones del menú anterior;

```
do {
    System.out.println("===== CINEPRIMI MANAGER =====");
    System.out.println("1. Listar películas");
    System.out.println("2. Añadir película");
    System.out.println("3. Eliminar película");
    System.out.println("4. Modificar película");
    System.out.println("5. Listar directores");
    System.out.println("6. Listar géneros");
    System.out.println("7. Listar valoraciones");
    System.out.println("8. Añadir valoración");
    System.out.println("9. Búsqueda avanzada");
    System.out.println("10. Top películas valoradas");
    System.out.println("0. Salir");

    Integer opcionLeida = LeerEntero("Seleccione una opción: ");

    if (opcionLeida == null) {
        continue;
    }

    opcion = opcionLeida;
```

y cada respuesta es un case en un switch que deriva a las funciones según la estructura planteada al principio: las que interactúan con el usuario para pedir datos, se definen en el mismo archivo Menu.java y tras conseguir esos datos, se pasan a la función DAO asociada. Por ejemplo insertar película pide todos los datos necesarios y una vez recopilados los pasa a la función insertarPeliculaCompleta en PeliculaDAO.java

Las que solo implican mostrar la query a la BD, llaman directamente a la función asociada al archivo DAO.

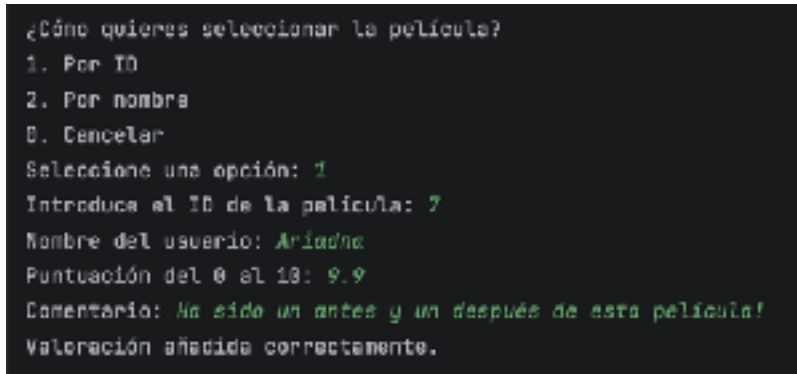
```
switch (opcion) {
    case 1 -> peliculaDAO.listar();
    case 2 -> insertarPelicula();
    case 3 -> eliminarPelicula();
    case 4 -> modificarPelicula();
    case 5 -> directorDAO.listar();
    case 6 -> generoDAO.listar();
    case 7 -> valoracionDAO.listarValoraciones();
    case 8 -> añadirValoracion();
    case 9 -> busquedaAvanzada();
    case 10 -> peliculaDAO.topPelículasValoradas();
    case 0 -> System.out.println("Saliendo...");
    default -> System.out.println("Opción no válida.");
}
```

Las opciones fáciles fueron género y directores, ya que en el archivo DAO solo tienen un método de listar con una query sencilla que fue:

- Género: `SELECT id_genero, nombre FROM géneros ORDER BY id_genero`
 - Director: `SELECT id_director, nombre, nacionalidad FROM directores ORDER BY id_director`
- Para añadir género y directores lo focalizo a través de insertar película

Se comienza a complicar en ValoracionesDAO, ya que no solo hay una query para listar las valoraciones existentes, sino que se pueden añadir valoraciones al insertar una película y decidí añadir la opción extra de valorar directamente una película ya existente en la lista.

Para evitar errores al entrar datos, se puede seleccionar la película por el nombre o por el ID y se muestra un listado de las películas para poder seleccionar una.



```
¿Cómo quieres seleccionar la película?
1. Por ID
2. Por nombre
3. Cancelar
Seleccione una opción: 1
Introduce el ID de la película: 7
Nombre del usuario: Ariadna
Puntuación del 0 al 10: 9.9
Comentario: Ha sido un antes y un después de esta película!
Valoración añadida correctamente.
```

Esto implica un método en Menu.java para recopilar la información (`anadirValoracion()`) y que otro método concreto para buscar la película por el nombre trabaje desde `ValoracionDAO` y lo asocie al id (`buscarIdPeliculaPorNombre()`) y que todos estos datos vayan a `insertarValoracion()` para modificar la tabla. Además de dos métodos adicionales que he necesitado reusar en otros métodos como: `leerEntero()` y `leerDouble()` para las valoraciones.

Por último, lo que ha sido todo un reto y la mayor parte de aprendizaje, ha sido configurar las películas de la manera que tenía en mente. Los métodos en `PeliculasDAO.java` y `Menu.java` han requerido de búsqueda externa a través de cursos de sql en Udemy, repaso de apuntes de primero, y miles de consultas a la IA para clarificar las dudas.

- Al listar las películas existentes, quería enlazar con toda la info disponible: director, género y valoraciones, por lo que la query es bastante más compleja. Tuve que crear métodos específicos para estas consultas como (`existePelicula()` o `existeDirector()`).
- El método de eliminar no tenía mucha complicación.
- Modificar me supuso también algo de búsqueda para completarla sin errores la recopilación de datos y la query.
- La búsqueda avanzada fue un reto para conseguir buscar filtrando por genero, acotar años y puntuación mínima es una parte que no ha quedado como quería y que todavía me da errores en algunos casos.
- La opción de Top 10 películas incluye una query para unir información de titulo y año de la película, con el director, con la valoración media y número de valoraciones.
- La opción de Insertar nuevas películas fue también muy compleja:
 - Crear un método en `Menu.java` para recoger los datos de la película, director, genero y valoración.
 - En `PeliculaDAO` lanzar queries simultaneas de `INSERT` con los datos recopilados.

A todo esto ir apagando fuegos de errores en la inserción de datos, código que peta y sale del programa por errores no detectados, queries que no funcionaban, errores tontos de gramática, etc...

Independientemente del resultado final, ha sido un proceso super interesante y enriquecedor.